

Lab 4: DNS Foundations with Cisco IOS and Wireshark

1. Background

The Domain Name System (DNS) is a critical Internet service that translates human-readable domain names (such as `www.example.local` or `www.colostate.edu`) into numerical IP addresses, allowing computers to locate each other on a network. Sockets provide the foundational programming interface that applications use to communicate over a network via TCP or UDP.

In enterprise and service provider networks, routers frequently act as local DNS servers, caching forwarders, and default gateways for connected client subnets. Modern network operating systems contain built-in DNS server subsystems capable of resolving local hostnames authoritatively from internal host tables and forwarding recursive queries to upstream name servers.

In this lab, you will deploy a client-server topology in Cisco Modeling Labs (CML) consisting of an **Alpine Linux Client** and a **Cisco IOS Router** acting as the authoritative DNS server. You will configure the router's DNS database, generate DNS query traffic using `nslookup`, capture the transactions in CML, and analyze the underlying DNS protocol structure in Wireshark.

AI Policy & Acceptable Use Guidelines



AI Policy & Acceptable Use Guidelines - Yellow Icon

Artificial Intelligence tools can be powerful learning aids, but they must be used ethically and responsibly to ensure you develop core networking skills.

- **Lab Assignments:** Do not use AI to perform lab steps or write your lab submissions. However, if you get stuck on a concept or technical obstacle, you may use AI as a virtual tutor to ask clarifying questions and help you troubleshoot.

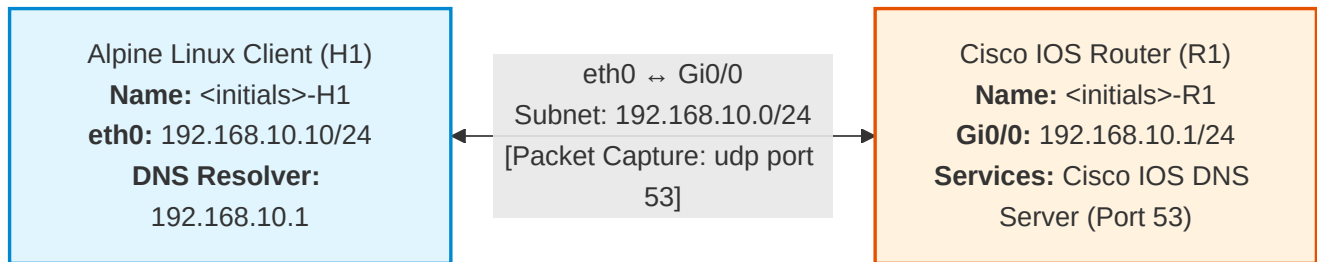
2. Objective / Purpose

The objective of this lab is to observe and analyze Domain Name System (DNS) protocol mechanics and socket resolution in an emulated network environment.

Specifically, you will:

- Deploy an **Alpine Linux Client** and a **Cisco IOS Router** in CML.
- Configure Cisco IOS native DNS server services (`ip dns server`) and authoritative static host mappings (`ip host`).
- Configure the Alpine Linux client interface and resolver configuration (`/etc/resolv.conf`).
- Execute DNS queries using `nslookup` to resolve local hostnames.
- Capture DNS query and response packets on the link directly via CML.
- Analyze DNS message formats, transaction IDs, header flags (QR, AA, RD, RA), port numbers, and Resource Records (RRs) in Wireshark.

3. CML Topology and Configuration Procedure



IP Addressing Table

Device	Node Type	Interface	IP Address	Subnet Mask	Default Gateway / DNS
Client (H1)	Alpine Linux	eth0	192.168.10.10	255.255.255.0 (/24)	192.168.10.1
Router (R1)	Cisco IOSv / IOL	GigabitEthernet0/0	192.168.10.1	255.255.255.0 (/24)	N/A (DNS Server)

Step 1: Deploying the Nodes in CML

1. Log in to the CML UI and create a new lab called `Lab_04_DNS_Foundations` .
2. Drag one **Alpine Linux** node onto the workspace and rename it `<initials>-H1` .
3. Drag one **Cisco IOSv** (or **IOSvL2 / IOL**) router node onto the workspace and rename it `<initials>-R1` .
4. Connect a link from `eth0` on H1 to `GigabitEthernet0/0` (or `Ethernet0/0`) on R1.
5. Click the **Start Lab** button (play icon) to power on both nodes.

Step 2: Configuring the Client Host (H1)

1. Click on the **H1** client node and navigate to the **Console** tab (or use the **Edit Config** tab before booting).
2. Enter the following commands to configure the static IP address, start the interface, and set the default DNS nameserver:

```
# Configure network interface eth0
ip addr add 192.168.10.10/24 dev eth0
ip link set eth0 up
ip route add default via 192.168.10.1

# Configure DNS resolver to point to the Cisco Router
echo "nameserver 192.168.10.1" > /etc/resolv.conf
```

Step 3: Configuring the Cisco IOS DNS Server (R1)

[!IMPORTANT] Action Required - Customize DNS Records with Your Last Name: The configuration below uses `haefner` solely as an illustrative example. **You MUST edit the commands to replace `haefner` with your own actual last name** (e.g., `smith.local` , `gonzalez.local` , `chen.local`):

- `ip domain name <lastname>.local`
- `ip dns primary <lastname>.local soa ns1.<lastname>.local admin.<lastname>.local`

- `ip host ns1.<lastname>.local 192.168.10.1`
- `ip host www.<lastname>.local 192.168.10.100`
- `ip host mail.<lastname>.local 192.168.10.20`

Note: Submitting screenshots, router host tables, or packet captures containing `haefner` rather than your own last name will result in a point deduction.

[!NOTE] **DNS Naming Rules & Allowed Characters (RFC 1035 / RFC 1123):** When formatting `<lastname>.local` :

- **Allowed Characters:** Letters (`a-z`), numbers (`0-9`), and hyphens (`-`). DNS is case-insensitive.
- **Prohibited Characters:** Spaces, underscores (`_`), apostrophes, dots within a label, and special symbols (`@` , `#` , `$` , `!`) are **invalid** in DNS hostnames.
- **Hyphen Rule:** A domain label cannot start or end with a hyphen (e.g., `-haefner-` is invalid).
- If your last name contains spaces or special characters (e.g., `St. John` or `O'Connor`), strip them and use clean alphanumeric characters (e.g., `stjohn.local` or `oconnor.local`).

1. Click on the **R1** router node and navigate to the **Console** tab.
2. Press **Enter** to access the prompt, then enter privileged EXEC and global configuration mode:

```
enable
configure terminal
hostname R1

! Configure interface IP address and bring interface UP
interface GigabitEthernet0/0
ip address 192.168.10.1 255.255.255.0
no shutdown
exit

! Enable DNS resolution and the built-in DNS Server daemon
ip domain lookup
ip domain name haefner.local
ip dns server
ip dns primary haefner.local soa ns1.haefner.local admin.haefner.local

! Define static Authoritative DNS Host Mappings (A-Records) -- REPLACE 'haefner' with YOUR
last name!
ip host ns1.haefner.local 192.168.10.1
ip host www.haefner.local 192.168.10.100
ip host mail.haefner.local 192.168.10.20
ip host cml-router.internal 192.168.10.1
ip host www.colostate.edu 129.82.100.10

end
```

3. Verify the static host mappings on the router:

```
show hosts
```

What you should see: The table will display your configured `<lastname>.local` hostnames and their mapped IP addresses with the `Flags: perm, OK` attributes.

4. Lab Execution and Traffic Generation

[!IMPORTANT] REMINDER: All screenshots, console outputs, and packet captures (.pcap files) must be taken from the SAME active lab session. Mixing screenshots or trace files across different runs or sessions is not permitted.

Step 1: Starting the Packet Capture in CML

1. In the CML canvas, click on the **link** connecting H1 and R1.
2. In the bottom pane, navigate to the **Packet Capture** tab.
3. Set the packet filter to `udp port 53` (or leave blank to capture all traffic).
4. Click **Start** to begin capturing traffic on the link.

Refer to the image below for how to configure the packet capture filter in the CML interface:

IOSV-0 ALPINE-0-E..THERNET0/0 ALPINE-0

CANCEL APPLY

Max Packets:
5000

Max Time:
10

BPF Templates: BPF:
Selecting a template will populate the bpf filter to the right
udp port 53

Encapsulation:
Ethernet
Select the encapsulation (typically Ethernet)

Step 2: Performing DNS Queries from Client (H1)

Open the **Console** tab on H1 and execute the following DNS queries (replacing `haefner` with your configured `<lastname>` domain):

1. Query 1: Local Web Hostname Resolution

```
nslookup www.haefner.local 192.168.10.1
```

- **What you should see:** The server returns the A-record address `192.168.10.100`.
- *If you don't see this:* Verify that `ip dns server` is active on R1 and interface `Gi0/0` is UP (`ping 192.168.10.1`).

2. Query 2: Mail Server Resolution

```
nslookup mail.haefner.local 192.168.10.1
```

- **What you should see:** The server returns `192.168.10.20`.

3. Query 3: Querying Additional Authoritative Records

```
nslookup www.colostate.edu 192.168.10.1
```

- **What you should see:** The router returns the mapped IP address `129.82.100.10`.

4. Query 4: Resolving Name via Default Resolver Configuration

```
nslookup ns1.haefner.local
```

- **What you should see:** The client queries the default nameserver configured in `/etc/resolv.conf` (`192.168.10.1`) and resolves `ns1.haefner.local` to `192.168.10.1`.

*[!NOTE] **Protocol Inspection via Packet Capture:** Alpine Linux uses the standard, lightweight `nslookup` utility for host resolution. While command-line tools such as `dig` format raw DNS protocol fields directly on the terminal, you will inspect the exact wire-level protocol fields—such as the **Header Flags** (`QR` , `AA` , `RD` , `RA`), **Transaction IDs**, and Resource Record **TTL** values—directly from the captured packet trace (`dns_queries.pcap`) in Wireshark (Section 5).*

Step 3: Downloading the Packet Capture

1. In CML, go to the link's **Packet Capture** tab and click **Stop**.
2. Click the **Download** button to save the `.pcap` capture file to your local computer. Name the file `dns_queries.pcap`.

5. Wireshark Analysis and Questions

Open `dns_queries.pcap` in Wireshark and answer the following questions:

IMPORTANT - Annotation Requirement: For questions 2 and 5, you must include an **annotated screenshot** of the relevant packets in Wireshark (using arrows, highlights, or boxes) pointing out the specific fields, flags, and transaction IDs you are describing.

5.1 DNS Query Message Analysis

1. Filter your capture for `dns`. Locate the first DNS query packet sent for `www.<lastname>.local`. What is the packet number in your trace?
2. What transport layer protocol (TCP or UDP) is used to carry the DNS query? What is the source port and destination port? Provide an **annotated screenshot** showing the transport layer header.
3. What is the **Transaction ID** of the query? Why is a transaction ID necessary in UDP-based DNS communications?
4. In the DNS header, expand the **Flags** field. What is the value of the **QR (Query/Response)** bit and the **RD (Recursion Desired)** bit? What do these bits signify?
5. Under the **Queries** section of the query packet, what is the queried **Domain Name, Type** (e.g., `A`, `AAAA`, `NS`), and **Class** (e.g., `IN`)?

5.2 DNS Response Message Analysis

6. Locate the corresponding DNS response message sent from the Cisco router back to the client. What is the packet number?
7. Compare the **Transaction ID** of the response message to the transaction ID of the original query message. Do they match? Explain why this is critical for the client resolver.
8. Expand the **Flags** field in the response message:

- What is the value of the **QR (Response)** bit in the response?
 - Is the **Authoritative Answer (AA)** flag set to 1? Explain why the Cisco router is authoritative for this query.
 - Is the **Recursion Available (RA)** flag set or unset?
 - What is the value of the **Response Code (RCODE)** (e.g., No error (0))?
9. Expand the **Answers** section of the response message:
- How many Resource Records (RRs) are returned?
 - What is the resolved IP address for `www.<lastname>.local` ?
 - What is the Time-To-Live (TTL) value reported in the answer Resource Record? What is the purpose of the TTL in DNS caching?

5.3 Architectural & Protocol Questions

10. In Cisco IOS, what is the role of the `ip dns server` command versus the `ip host` command?
11. When would a DNS client or server use **TCP Port 53** instead of **UDP Port 53**? (Describe at least two distinct scenarios and explain why TCP is required instead of UDP).
12. What is the difference between an **Authoritative DNS Server** and a **Recursive / Caching DNS Resolver**?

6. Troubleshooting

Issue Encountered: [Describe what went wrong, if anything]

Discovery Method: [How did you identify the issue?]

Resolution: [How did you resolve it?]

7. What to Hand In

Submit a lab report containing:

1. **CML Topology Screenshot:** A screenshot of your CML workspace showing the connected `<initials>-H1` client and `<initials>-R1` Cisco router.
2. **Cisco Router Configuration Screenshot:** Console output showing `show hosts` and `show running-config | include ip (dns|host|domain)` on R1.
3. **Client Console Screenshot:** Terminal output showing the successful results of your `nslookup` commands on H1.
4. **Downloaded .pcap Capture File:** The `dns_queries.pcap` packet trace file.
5. **Annotated Wireshark Screenshots:**
 - Annotated screenshot of the DNS Query packet showing UDP ports, Transaction ID, and Question section (Question 2).
 - Annotated screenshot of the DNS Response packet showing matching Transaction ID, AA flag, and Answer section (Question 8).
6. **Written Answers:** Complete answers to the 12 analysis questions in Section 5.